

COUPLING IN SOFTWARE DESIGN

La entrevista analiza el concepto de **acoplamiento (coupling)** en el diseño de software a partir de un podcast con Vlad Khononov, destacando su importancia para **gestionar la complejidad** en los sistemas.

Primero, se explica qué es la **complejidad** usando el marco Cynefin: un sistema es complejo cuando no podemos predecir el resultado de un cambio sin probarlo. En software, esto es lo que se intenta evitar. Para ello, el acoplamiento se utiliza como herramienta para hacer los sistemas más predecibles.

Se diferencian dos tipos de complejidad:

- **Esencial:** propia del dominio (ej. sistemas médicos o financieros), no se puede eliminar.
- **Accidental:** causada por el diseño del software, y sí se puede controlar.

Aquí entra la **modularidad**, cuyo objetivo es reducir la complejidad. Un sistema modular permite:

- Saber fácilmente **dónde hacer cambios**.
- Entender **cómo afectarán esos cambios** al resto del sistema.

Un módulo se define por tres características:

1. Implementa una funcionalidad clara.
2. Tiene una interfaz pública.
3. Puede compilarse de forma independiente (al menos en teoría).

Además, los módulos existen en distintos niveles (clases, métodos, servicios...), formando una estructura **fractal**. Cada nivel ayuda a gestionar la complejidad mediante abstracciones.

El texto también explica que no siempre se puede lograr una modularidad perfecta. En la práctica, hay que equilibrar entre buen diseño y limitaciones reales, aceptando cierto grado de acoplamiento.

Tipos de acoplamiento

Khononov propone un modelo simplificado llamado **“fuerza de integración”**, con cuatro niveles:

- **Por contrato:** solo se comparten interfaces (mejor caso).
- **De modelo:** se comparten estructuras de datos.

- **Funcional:** se comparte lógica de negocio.
- **Intrusivo:** se accede a detalles internos (peor caso).

También menciona la **connascencia**, que permite analizar con más detalle las dependencias.

Factores clave del acoplamiento

El diseño se analiza con tres dimensiones:

1. **Conocimiento:** cuánta información comparten los módulos.
2. **Distancia:** qué tan separados están.
3. **Volatilidad:** con qué frecuencia cambian.

Relaciones importantes:

- **Poco conocimiento + gran distancia → buen diseño (modularidad).**
- **Mucho conocimiento + gran distancia → alta complejidad (problema).**
- **Valores opuestos → modularidad.**
- **Valores iguales → complejidad.**

La **volatilidad** modifica estas reglas:

- Si algo cambia mucho → se necesita mejor diseño (más modular).
- Si cambia poco → se puede tolerar más complejidad.

Se relaciona con los subdominios:

- **Core:** críticos y cambiantes.
- **Apoyo:** menos importantes.
- **Genéricos:** ya resueltos (ej. login).

Conclusión

El acoplamiento es inevitable en software, pero debe **gestionarse cuidadosamente**.

Entender cómo se relacionan los módulos (en conocimiento, distancia y cambio) permite crear sistemas más **mantenibles, escalables y predecibles**.